Borg IA Cube: sistema distribuido de CNN con chat multiplataforma y middleware RabbitMQ

Espinoza Benavente Hector
Inga Chávez Jorly
Villanueva Alzamora Farid

Universidad Nacional de Ingeniería – Facultad de Ciencias

CC4P1 – Programación Concurrente y Distribuida
Práctica Calificada 3 – 2026-II

Índice

1. Introducción **2**

2. Marco teórico **2**
2.1. Redes neuronales convolucionales . . . 2
2.2. Middleware de mensajería: RabbitMQ . . . 2
2.3. Sistemas distribuidos y sockets . . . 3
2.4. Concurrencia y sincronización . . . 3
2.5. Transacciones en bases de datos . . . 3
2.6. Desempeño . . . 3

3. Metodología e implementación **3**
3.1. Arquitectura del sistema . . . 3
3.2. Modelos de IA y *micro chunks* . . . 6
3.3. Protocolo de comunicación . . . 7
3.4. Bases de datos heterogéneas . . . 8
3.5. Conceptos de concurrencia en el código . . . 9
3.6. Despliegue . . . 9
3.7. Diseño experimental . . . 10

4. Resultados y discusión **11**
4.1. Métricas de los modelos . . . 11
4.2. Entrenamiento en las 4 configuraciones . . . 12
4.3. Inferencia en las 4 configuraciones . . . 13
4.4. Prueba de saturación . . . 14
4.5. 1000 registros y validación de las bases . . . 15
4.6. Discusión general . . . 15

5. Conclusiones **15**

6. Anexo de código fuente **16**

7. Anexo de documentación **16**

1. Introducción

Borg IA Cube continúa el concepto de *Open AI Cubes* del examen parcial: un sistema modular en el que cada "cubo" aporta una capacidad de inteligencia artificial. En esta práctica el usuario envía la foto de una célula a través de un **chat** (Android, escritorio o web) y recibe el diagnóstico de una red neuronal convolucional (CNN). Hay dos cubos: el **modelo A** detecta malaria (célula parasitada o no infectada) y el **modelo B** clasifica 8 tipos de células sanguíneas.

A diferencia del parcial, los nodos ya no se hablan directamente. Los clientes se conectan al *backend* con **sockets TCP propios**, y el backend, los nodos de IA y los nodos de base de datos se comunican solo a través de **RabbitMQ**, que actúa como *middleware* de mensajería asíncrona. El sistema usa **tres bases de datos de motores distintos** (PostgreSQL, MariaDB y MySQL): dos guardan los resultados de cada modelo y la tercera valida clientes, productos e imágenes.

Las CNN están escritas desde cero en Go, sin TensorFlow ni PyTorch. El sistema se desplegó con el servidor en **Google Cloud** y una laptop por integrante, cada una con su base de datos y su nodo de IA, conectadas por internet desde tres casas distintas.

Los objetivos de este trabajo son:

- Implementar una CNN con *micro chunks* de convolución y de MLP, entrenada con *datasets* elegidos por el grupo, en las configuraciones secuencial, paralela, distribuida y paralela distribuida.
- Integrar clientes Android, de escritorio y web, un *backend*, nodos de IA y tres bases de datos heterogéneas mediante sockets propios y RabbitMQ.
- Mantener consistentes las bases de datos frente a escrituras concurrentes.
- Evaluar el desempeño y saturar el sistema con 1000 transacciones por cliente.

2. Marco teórico

2.1. Redes neuronales convolucionales

Una CNN es una arquitectura de aprendizaje profundo pensada para datos en forma de cuadrícula, como las imágenes. Tiene una parte convolucional, que extrae características, y un perceptrón multicapa (MLP), que clasifica a partir de ellas.

2.1.1. Convolución, ReLU y *pooling*

Una convolución desliza filtros pequeños (aquí de 3×3) sobre la imagen. Cada filtro produce un mapa de activación que responde a un patrón local, como un borde o una textura:

$$\text{out}[f][y][x] = b_f + \sum_{c} \sum_{k_y} \sum_{k_x} W_{f,c,k_y,k_x} \, \text{in}[c][y + k_y - 1][x + k_x - 1].$$

La función ReLU, $\max(0, z)$, introduce no linealidad, y el *max pooling* de 2×2 reduce a la mitad cada dimensión espacial conservando el valor máximo de cada ventana.

2.1.2. Perceptrón multicapa y softmax

El mapa final se aplana en un vector que pasa por capas densas. La última produce un valor por clase, que la función softmax convierte en probabilidades. El entrenamiento minimiza la entropía cruzada con descenso de gradiente estocástico (SGD) con *momentum*.

2.2. Middleware de mensajería: RabbitMQ

RabbitMQ es un *broker* de mensajes que implementa el protocolo AMQP. Los productores publican mensajes en un *exchange*, que los enruta a colas; los consumidores leen de esas colas a su propio ritmo. Así se desacoplan las etapas: si un consumidor es lento, los mensajes esperan en la cola en lugar de perderse. Tres características de RabbitMQ son clave en este sistema:

- **Reparto entre consumidores:** si varios nodos consumen la misma cola, RabbitMQ reparte los mensajes entre ellos, lo que permite agregar nodos de IA sin cambiar nada más.
- ***Prefetch* y confirmación (*ack*):** cada consumidor recibe a la vez como máximo *prefetch* mensajes sin confirmar. Un mensaje no confirmado se reentrega si el consumidor cae.
- **RPC:** quien pregunta publica con una cola de respuesta (`reply_to`) y un identificador (`correlation_id`), y quien responde publica en esa cola con el mismo identificador.

2.3. Sistemas distribuidos y sockets

Un sistema distribuido está formado por procesos autónomos que se comunican por red para cumplir una tarea común. La comunicación con los clientes usa *sockets* TCP: cada socket es un extremo de un canal bidireccional entre dos procesos, y TCP garantiza la entrega confiable y en orden.

2.4. Concurrencia y sincronización

Cuando varios hilos acceden a datos compartidos sin coordinación ocurre una *condición de carrera*: el resultado depende del orden, no determinista, en que el sistema operativo intercala los hilos. Las herramientas usadas para evitarla son:

- **Exclusión mutua:** `ReentrantLock`, `synchronized` y `sync.Mutex`.
- **Estructuras concurrentes:** `ConcurrentHashMap`, `LinkedBlockingQueue` y contadores atómicos.
- **Productor–consumidor:** colas bloqueantes y canales de Go.
- **Barreras:** un grupo de tareas espera a que todas terminen antes de seguir (`sync.WaitGroup`).
- **Hilos con estado propio:** cada hilo trabaja con su propia copia de los datos (por ejemplo, su propio búfer de gradiente), y al final se combinan.

2.5. Transacciones en bases de datos

Una transacción agrupa varias operaciones que se aplican todas o ninguna (atomicidad). Para evitar que dos transacciones concurrentes lean el mismo valor y lo sobrescriban, `SELECT ... FOR UPDATE` bloquea la fila hasta que termina la transacción. Una inserción **idempotente** (`ON CONFLICT DO NOTHING` en PostgreSQL, `INSERT IGNORE` en MySQL y MariaDB) no duplica el registro si el mismo mensaje llega dos veces.

2.6. Desempeño

Las métricas usadas son:

- **Latencia:** tiempo entre que un cliente envía una consulta y recibe la respuesta.
- **Throughput:** transacciones procesadas por segundo.
- **Speedup:** $S = T_{\text{secuencial}}/T$.
- **Costo de paralelización:** el tiempo que no es cómputo útil (enviar datos, esperar y combinar resultados). Es lo que impide que S crezca de forma lineal.

3. Metodología e implementación

3.1. Arquitectura del sistema

La Figura 1 muestra la arquitectura. Los clientes y los nodos siempre abren la conexión hacia el servidor: los clientes, al backend (puerto 5000) o al servidor web (puerto 8080), y los nodos, a RabbitMQ (puerto 5672). Por eso no hace falta abrir puertos en las casas de los integrantes.

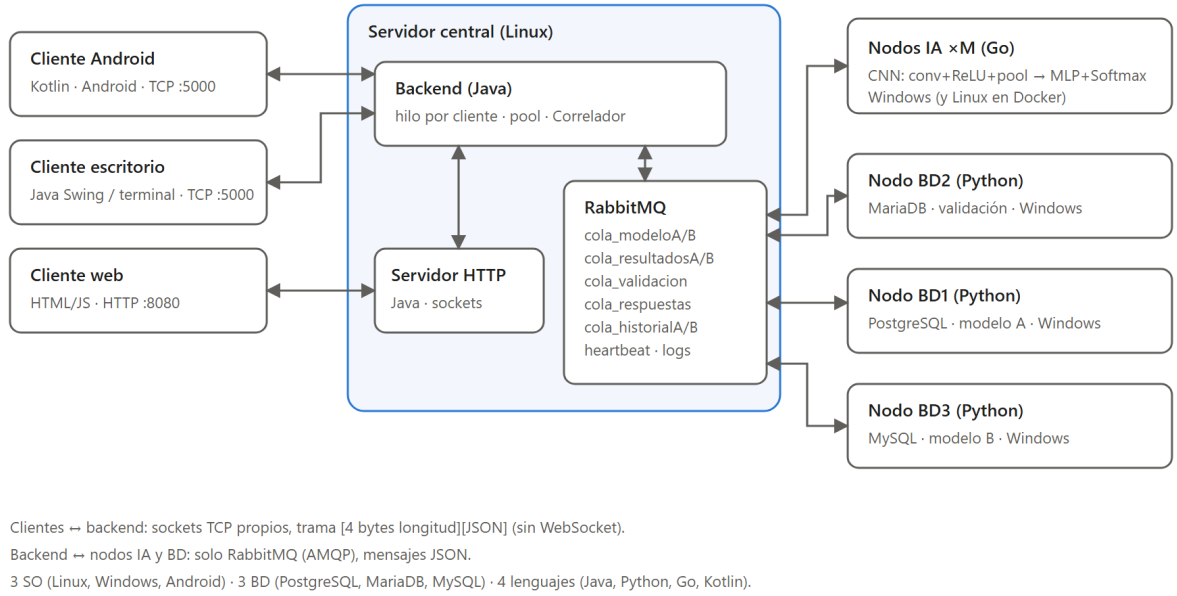


Figura 1: Arquitectura de Borg IA Cube: clientes, servidor central con RabbitMQ, nodos de IA y nodos de base de datos.

3.1.1. Módulos

- **Clientes:** app Android en Kotlin (Figura 2), chat de escritorio en Java Swing con una versión de terminal, y chat web en HTML/JS. Los tres permiten iniciar sesión, registrarse, enviar una imagen, ver el historial y lanzar una prueba de estrés.
- **Servidor web:** servidor HTTP propio, hecho con `ServerSocket` de Java. Sirve el chat web y reenvía cada acción del navegador al backend por TCP. No usa WebSocket.
- **Backend:** recibe a los tres clientes, valida usuarios contra BD2, enruta las imágenes a RabbitMQ y devuelve la respuesta al cliente correcto.
- **Nodos de IA:** ejecutan la CNN sobre las imágenes de `cola_modeloA` y `cola_modeloB`, y atienden las tareas del entrenamiento distribuido.
- **Nodos de BD:** BD2 valida; BD1 y BD3 guardan los resultados de los modelos A y B.

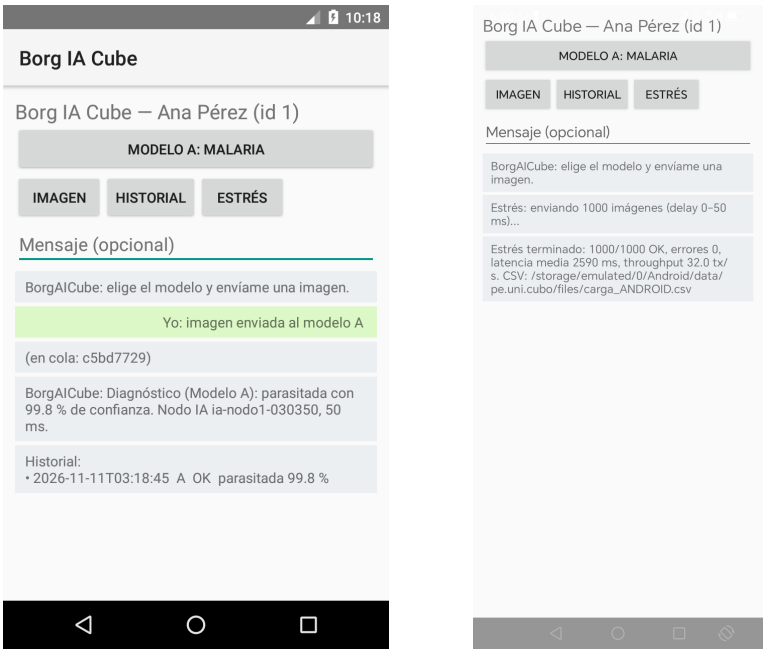


Figura 2: App Android. Izquierda: diagnóstico de una célula parasitada (99,8 %) e historial. Derecha: prueba de estrés de 1000 imágenes contra el servidor en Google Cloud, sin errores.

3.1.2. Componentes del código

Componente	Responsabilidad
`java/src/cubo/backend`	`Backend`, `ManejadorCliente`, `PuenteRabbit` (topología, publicador y RPC), `Correlador` y `Sesion`.
`java/src/cubo/web`	Servidor HTTP propio (`ServidorHttp`, `PeticionHttp`, `RespuestaHttp`).
`java/src/cubo/desktop`	Chat Swing (`ClienteDesktop`) y de terminal (`ClienteTerminal`).
`java/src/cubo/carga`	`GeneradorCarga`: consultas o registros con *delay* aleatorio.
`go/cnn`	CNN, *backpropagation*, dataset, preprocesamiento y entrenamiento paralelo.
`go/distrib, go/rmq`	Entrenamiento distribuido por RabbitMQ; topología, heartbeat y métricas.
`go/cmd/nodo, go/cmd/entrenar`	Nodo de IA y entrenamiento en las 4 configuraciones.
`python/`	`nodo_bd.py` (BD1, BD2, BD3), `validador.py`, `monitor.py`, `verificar_consistencia.py`.
`android/`	`Conexion.kt` (socket y protocolo) y `MainActivity.kt`.
`sql/`	Scripts de creación de BD1, BD2 y BD3.

Tabla 1: Componentes principales de la implementación.

3.2. Modelos de IA y *micro chunks*

3.2.1. Datasets

	Modelo A: malaria	**Modelo B: células sanguíneas**
Fuente	NIH, *Malaria Cell Images* [2]	BloodMNIST, de MedMNIST v2 [3], derivado de [4]
Imágenes	27 558 (13 779 por clase)	17 092
Clases	parasitada, no infectada	basófilo, eosinófilo, eritroblasto, granulocito inmaduro, linfocito, monocito, neutrófilo, plaqueta
Partición	22 046 / 2 754 / 2 758 (80/10/10, estratificada)	11 959 / 1 712 / 3 421 (partición oficial)
Preprocesamiento	Relleno a cuadrado con negro, redimensión bilineal a 32×32, RGB	Imágenes de 28×28 RGB

Tabla 2: Datasets elegidos por el grupo (entrenamiento / validación / prueba).

Los datasets se guardan en un formato binario propio: una cabecera `DSET` con N, C, H, W y K, y por imagen `[partición][etiqueta][C×H×W bytes]`. Cada canal se normaliza con la media y la desviación del set de entrenamiento. El nodo de IA aplica el mismo relleno, redimensión y normalización a las fotos que llegan por el chat.

3.2.2. Arquitectura de la red

Micro chunk	**Capa**	**Entrada (A)**	**Salida (A)**	**Detalle**
Convoluciones	conv1 + ReLU	$3 \times 32 \times 32$	$8 \times 32 \times 32$	filtro 3×3, padding 1
	pool1	$8 \times 32 \times 32$	$8 \times 16 \times 16$	máximo 2×2
	conv2 + ReLU	$8 \times 16 \times 16$	$16 \times 16 \times 16$	filtro 3×3, padding 1
	pool2	$16 \times 16 \times 16$	$16 \times 8 \times 8$	máximo 2×2
MLP	fc1 + ReLU	1 024	64	densa
	fc2 + softmax	64	K	una salida por clase

Tabla 3: Capas de la CNN: 67 122 pesos en el modelo A y 52 152 en el modelo B.

El *backpropagation* se escribió a mano. Una prueba automática (`go test ./cnn`) compara el gradiente analítico con el numérico en pesos al azar.

3.2.3. Entrenamiento paralelo: un búfer por hilo y una barrera por lote

Cada lote de 32 imágenes se reparte entre H goroutines. Cada una acumula el gradiente en *su propio* búfer, así que no hay escrituras compartidas ni condición de carrera. Un `sync.WaitGroup` hace de barrera y luego se suman los búferes y se actualizan los pesos:

```
func (e *Entrenador) Paso(d *Dataset, lote []int) (perdida float64, aciertos int) {
    var wg sync.WaitGroup
    for h := 0; h < e.Hilos; h++ {
        wg.Add(1)
        go func(h int) {
            defer wg.Done()
            g := e.grads[h]                              // búfer propio de la goroutine
            for j := h; j < len(lote); j += e.Hilos {  // reparto intercalado del lote
                e.Red.Muestra(d, lote[j], e.xs[h])
                e.Red.Gradiente(e.trabajos[h], e.xs[h], int(d.Etiq[lote[j]]), g)
            }
        }(h)
    }
    wg.Wait()   // barrera: todas terminaron su parte del lote
    // se suman los H búferes y se aplica SGD con momentum
    ...
}
```

3.2.4. Entrenamiento distribuido: promedio de modelos por RabbitMQ

El coordinador (`entrenar -modo dist`) publica en `cola_entrenamiento` una tarea por nodo, con los pesos actuales y las imágenes que le tocan. Cada nodo de IA hace 10 pasos de SGD sobre su parte, con 1 hilo (*dist*) o con todos sus núcleos (*pardist*), y devuelve sus pesos a la cola privada del coordinador. El coordinador los promedia ponderando por número de imágenes y empieza la siguiente ronda.

3.3. Protocolo de comunicación

3.3.1. Clientes y backend: sockets TCP propios

Cada mensaje es una trama `[4 bytes con la longitud N, big-endian][N bytes de JSON en UTF-8]`. El JSON se arma y se lee a mano. Una consulta lleva `idCliente`, `tipoConsulta`, la imagen en Base64 (`payload`) y `timestamp`:

```
-> {"tipo":"CONSULTA","token":"9f1c...","idCliente":1,"tipoConsulta":"IMAGEN","modelo":"A",
    "formato":"png","payload":"iVBORw0KGgo...","timestamp":1794367427500}
<- {"tipo":"ACK","idSolicitud":"5e0b...","estado":"EN_COLA"}
<- {"tipo":"RESULTADO","idSolicitud":"5e0b...","etiqueta":"parasitada","probabilidad
    ":0.998,
    "nodoIA":"ia-integrante02-...","latenciaTotalMs":1404}
```

3.3.2. Backend y nodos: colas de RabbitMQ

Cola	Consume	Contenido
`cola_modeloA / B`	Nodos de IA	`idSolicitud`, imagen, `modeloDestino`, `timestamp`
`cola_resultadosA / B`	BD1 / BD3	Lo anterior más etiqueta, probabilidades y nodo de IA
`cola_validacion`	BD2	Validaciones RPC: login, registro, solicitud e imagen
`cola_respuestas`	Backend	Resultado final de cada `idSolicitud`
`cola_historialA / B`	BD1 / BD3	Consultas del historial del usuario (RPC)
`cola_entrenamiento`	Nodos de IA	Tareas del entrenamiento distribuido
`cola_heartbeat, cola_logs, cola_metricas`	Backend, monitor	Control: nodos vivos, registros, CPU y memoria

Tabla 4: Colas de RabbitMQ (exchanges `ex_cubo` y `ex_control`).

3.3.3. Diagrama de protocolo

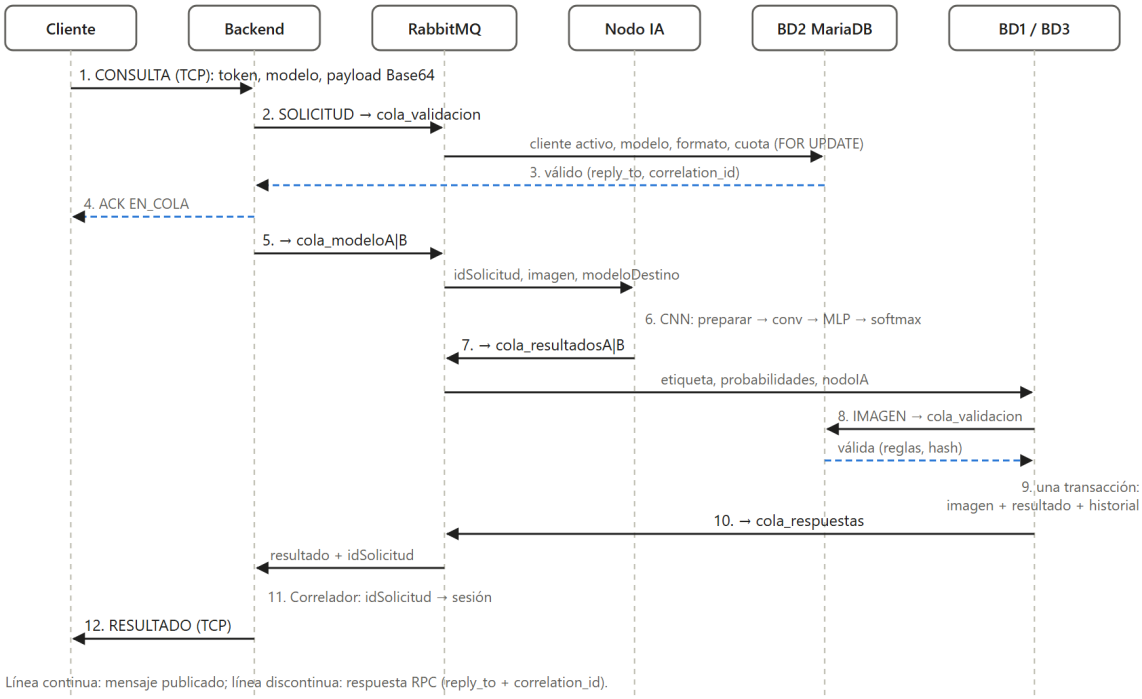


Figura 3: Flujo de una consulta: Cliente → Backend → RabbitMQ → Nodo IA → RabbitMQ → Nodo BD → RabbitMQ → Backend → Cliente.

BD2 valida cada consulta dos veces: antes de la IA (cliente activo, modelo habilitado, formato y cuota) y antes de guardar (reglas de la imagen y *hash* bloqueado). El backend usa el `Correlador` para saber a qué socket enviar cada resultado. Su `remove` sobre un `ConcurrentHashMap` es atómico: si la respuesta y el *timeout* llegan a la vez, solo uno de los dos "gana" y el cliente recibe un único mensaje.

3.4. Bases de datos heterogéneas

Base	Motor	Tablas	Rol
BD1	PostgreSQL 16	`imagenes`, `resultados`, `historial_consultas`, `metricas_entrenamiento`	Modelo A: metadatos, etiqueta, probabilidad, fecha e historial
BD2	MariaDB 11	`clientes`, `modelos`, `cliente_modelo`, `reglas_imagen`, `imagenes_bloqueadas`, `auditoria_validaciones`	Validación del cliente, del producto (modelo) y de la imagen
BD3	MySQL 8	Las mismas que BD1	Modelo B

Tabla 5: Bases de datos. Los scripts de creación están en `sql/`.

Cada nodo de BD atiende los mensajes con un *pool* de hilos y **una conexión por hilo**, nunca compartida. Imagen, resultado e historial se guardan en una sola transacción, y la cuota diaria del cliente se descuenta con `FOR UPDATE`:

```
def _solicitud(cur, msg):
    """Valida cliente activo, modelo habilitado, formato/tamaño y consume cuota (con FOR
        UPDATE)."""
    cur.execute("SELECT estado, cuota_diaria, usadas_hoy, fecha_cuota < CURDATE() FROM
        clientes "
                "WHERE id_cliente = %s FOR UPDATE", (id_cliente,))
    ...
    cur.execute("UPDATE clientes SET usadas_hoy = %s, fecha_cuota = CURDATE() WHERE
        id_cliente = %s",
```

```
7                (usadas + 1, id_cliente))
```

Prueba de concurrencia (`pruebas_concurrencia.py`)	**Resultado**
20 hilos insertan el mismo `id_solicitud` en BD1 y en BD3	1 fila por tabla
50 hilos piden validación para un cliente con cuota 10 en BD2	exactamente 10 aprobadas
20 hilos registran el mismo usuario nuevo en BD2	exactamente 1 creado

Tabla 6: Pruebas de escritura concurrente sobre las bases reales.

3.5. Conceptos de concurrencia en el código

Componente	**Hilos y sincronización**
Backend (Java)	Un hilo por conexión (`CachedThreadPool`) y un *pool* fijo para las consultas. Cada sesión escribe en su socket con un `ReentrantLock`, así las tramas no se intercalan. `ConcurrentHashMap` para sesiones y solicitudes pendientes; las publicaciones pasan por una `LinkedBlockingQueue` que vacía un único hilo publicador. Contadores con `AtomicLong`.
Servidor HTTP (Java)	*Pool* de 128 hilos; `ConcurrentHashMap` de sesiones web.
Nodo de IA (Go)	Productor–consumidor: RabbitMQ entrega hasta H mensajes (*prefetch*) y H goroutines los leen de un canal de Go. Publicador protegido con `sync.Mutex`; contadores `atomic.Int64`.
Nodos de BD (Python)	`ThreadPoolExecutor`, una conexión por hilo, transacciones, `FOR UPDATE`, inserciones idempotentes y `threading.Lock` para los contadores.
Android (Kotlin)	Hilo lector, hilo de *ping* y escritura del socket en `synchronized`; la interfaz solo se toca desde `runOnUiThread`.

Tabla 7: Concurrencia en cada componente.

3.6. Despliegue

3.6.1. Clúster del equipo

	Integrante01 (Farid)	**Integrante02 (Hector)**	**Integrante03 (Jorly)**
Base de datos	BD2 – MariaDB	BD1 – PostgreSQL	BD3 – MySQL
Nodos	BD2 (Python) + IA (Go)	BD1 (Python) + IA (Go) + carga	BD3 (Python) + IA (Go)
Procesador	Intel Core i7-1255U	Intel Core i5-13420H	AMD Ryzen 5 5500U
Núcleos / hilos	10 / 12	8 / 12	6 / 12
RAM	16 GB	32 GB	16 GB
Sistema operativo	Windows 11	Windows 11 (BD e IA en Docker)	Windows

Tabla 8: Laptops del clúster, cada una conectada desde una red doméstica distinta.

El cliente móvil corre en un celular Android. El sistema usa así **3 sistemas operativos** (Linux en el servidor, Windows en las laptops y Android en el celular), **3 motores de base de datos** en 3 nodos independientes y **4 lenguajes**: Java en el servidor, Python en los nodos de BD, Go en los nodos de IA y Kotlin en el cliente móvil. En la laptop de Integrante02, Windows no permite ejecutar binarios sin firmar, por lo que su base y su nodo de IA corren en contenedores Docker.

3.6.2. Servidor en Google Cloud

El servidor corre en la misma máquina virtual del parcial: `oac-servidor`, una instancia `e2-micro` con Ubuntu 24.04 LTS en la región `southamerica-west1` (Santiago de Chile), con IP estática. Ejecuta RabbitMQ, el backend y el servidor web. Una regla de firewall abre los puertos TCP 5000, 8080 y 5672. La máquina se prepara por SSH con `scripts/nube_preparar_vm.sh`, que instala RabbitMQ y Java, crea el usuario de RabbitMQ con una clave aleatoria y compila el proyecto. El backend y la web corren dentro de `tmux`, así siguen activos al cerrar el SSH (Figuras 4 y 5).

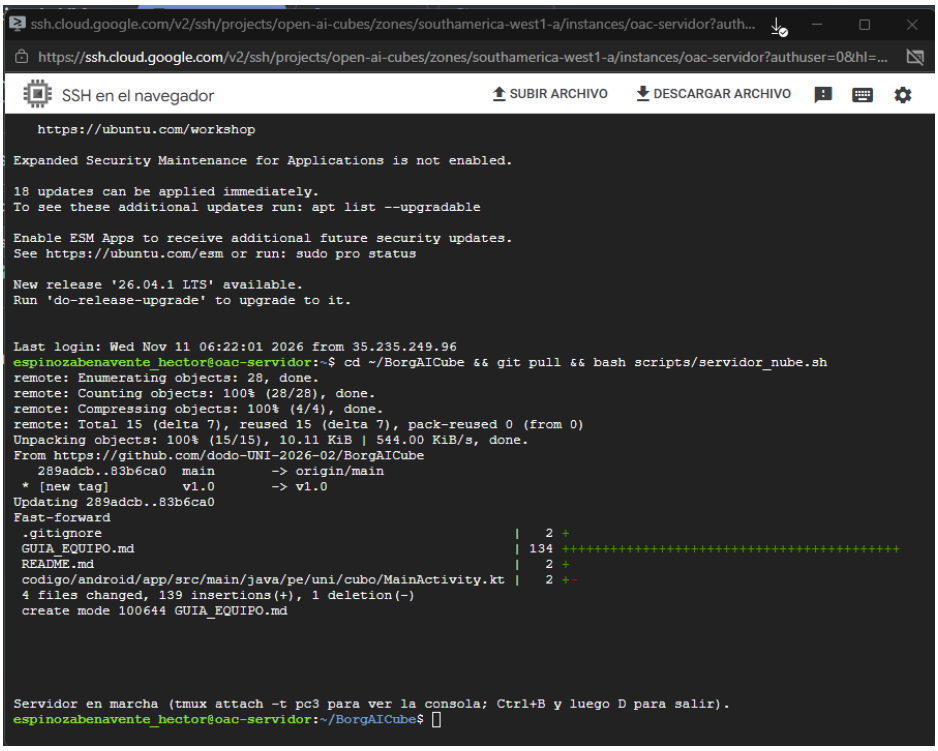


Figura 4: Arranque del servidor en la VM de Google Cloud por SSH desde el navegador.

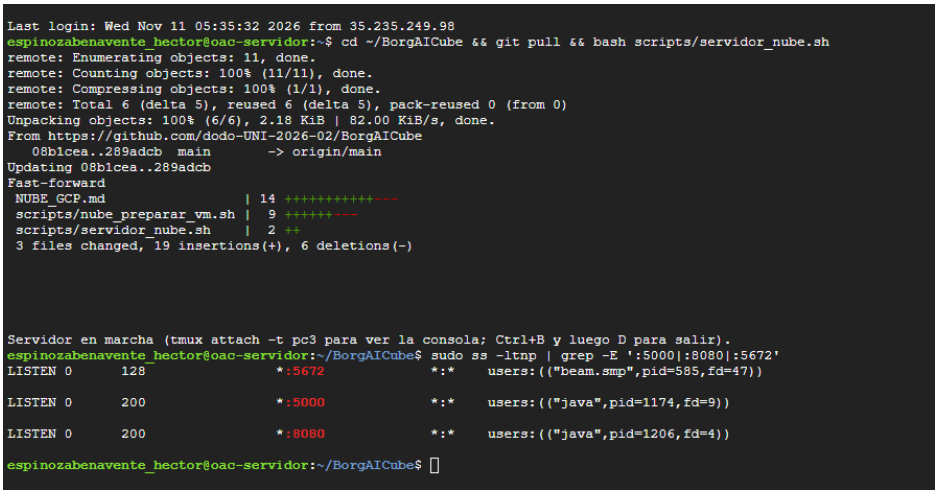


Figura 5: Procesos escuchando en la VM: RabbitMQ (5672), backend (5000) y servidor web (8080).

3.7. Diseño experimental

Las mediciones se hicieron el 11/11/2026 con el clúster real. La carga y el coordinador de entrenamiento se lanzaron desde la laptop de Integrante02. Para cambiar de configuración solo se reiniciaban los nodos de IA con otros parámetros:

Configuración	Nodos de IA	Experiencias
Secuencial	1 nodo, 1 núcleo y 1 hilo	Inferencia; entrenamiento de A y B
Paralelo	1 nodo, 12 hilos	Inferencia; entrenamiento de A y B
Distribuido	3 laptops, 1 hilo cada una	Inferencia; entrenamiento de A y B
Paralelo distribuido	3 laptops, 12 hilos cada una	Inferencia; entrenamiento de A y B; saturación; 1000 registros

Tabla 9: Configuraciones medidas.

- **Entrenamiento:** 3 épocas, lotes de 32, SGD con $\eta = 0{,}01$ y *momentum* 0,9, misma semilla en todas las corridas.
- **Inferencia:** 1000 consultas por el cliente de escritorio y 1000 por el web a la vez, con 32 sesiones y un *delay* aleatorio de 0 a 50 ms entre envíos.
- **Saturación:** escritorio, web y Android, 1000 transacciones cada uno, a la vez.
- **Instrumentación:** `monitor.py` registra cada 0,5 s la profundidad de las colas de RabbitMQ y la CPU y memoria que publica cada nodo. Todo se guarda en CSV, de donde salen las gráficas.

4. Resultados y discusión

4.1. Métricas de los modelos

Los modelos entregados se entrenaron 12 épocas en modo paralelo.

Modelo	Acc. entrenamiento	Acc. validación	Acc. prueba
A (malaria)	97,3 %	96,6 %	95,2 %
B (células sanguíneas)	94,4 %	90,0 %	88,6 %

Tabla 10: Accuracy de los modelos entregados.

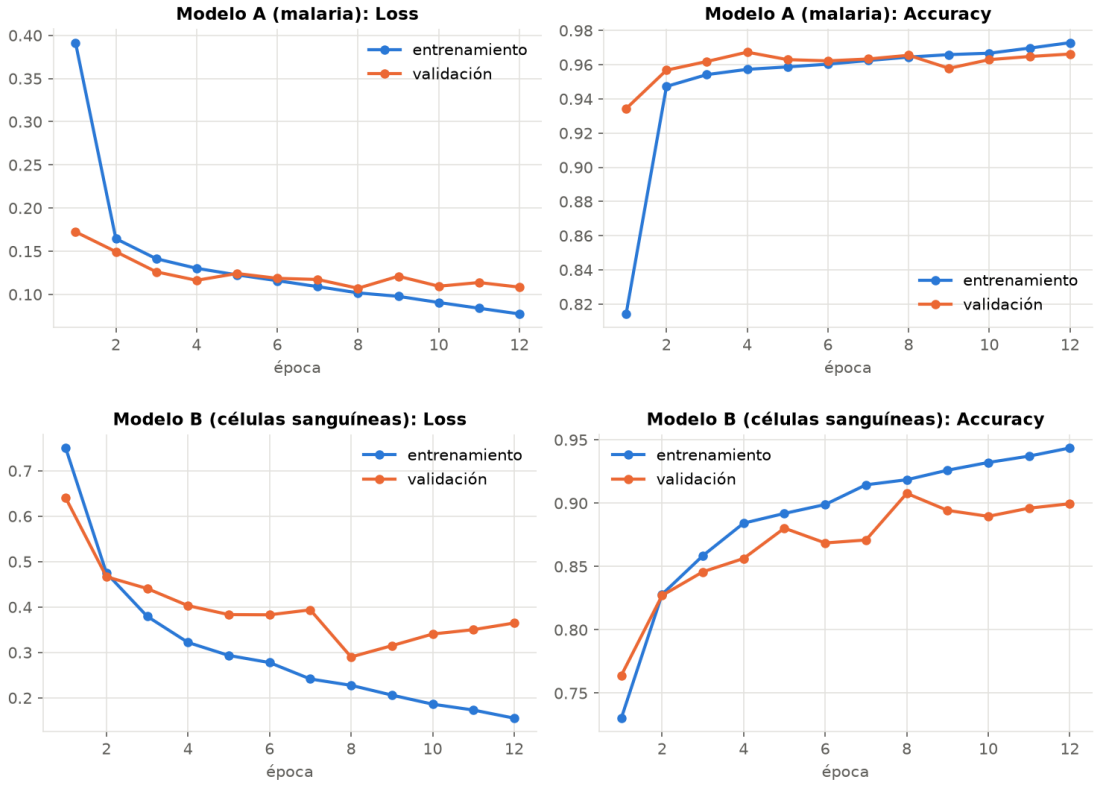


Figura 6: Loss y accuracy por época de los modelos A y B.

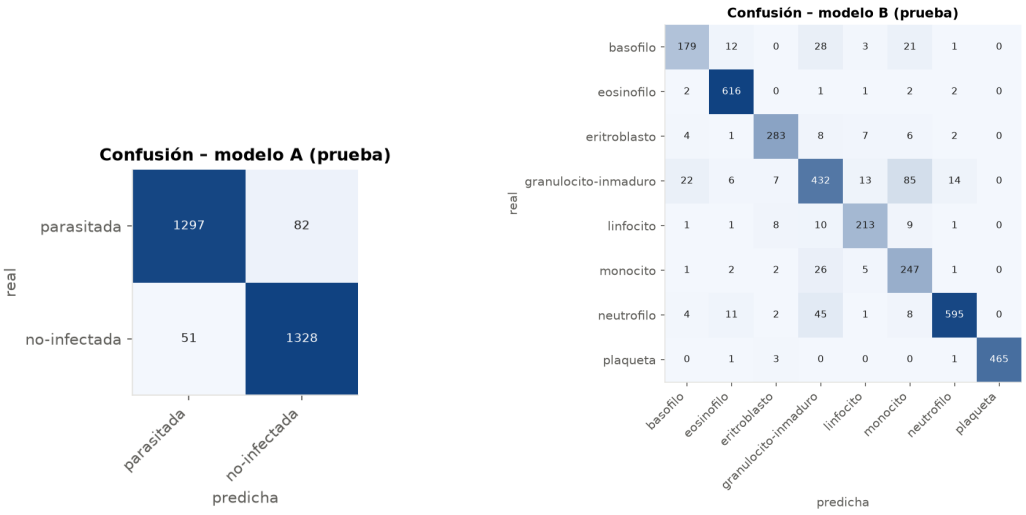


Figura 7: Matrices de confusión sobre el set de prueba.

En el modelo B, la mayoría de los errores mezclan granulocitos inmaduros con monocitos y neutrófilos, que son morfológicamente parecidos. Las plaquetas y los eosinófilos casi no se confunden.

4.2. Entrenamiento en las 4 configuraciones

		Modelo A			Modelo B		
Configuración	**Máq. × hilos**	**s/época**	**Speedup**	**Acc.**	**s/época**	**Speedup**	**Acc.**
Secuencial	1 × 1	38,4	1,00	95,3 %	16,6	1,00	83,7 %
Paralelo	1 × 12	6,4	6,00	95,0 %	2,8	5,88	83,9 %
Distribuido	3 × 1	56,3	0,68	82,3 %	26,3	0,63	80,2 %
Paralelo distribuido	3 × 12	30,1	1,27	83,4 %	16,6	1,00	80,6 %

Tabla 11: Entrenamiento con el clúster real (3 épocas). Speedup respecto al secuencial.

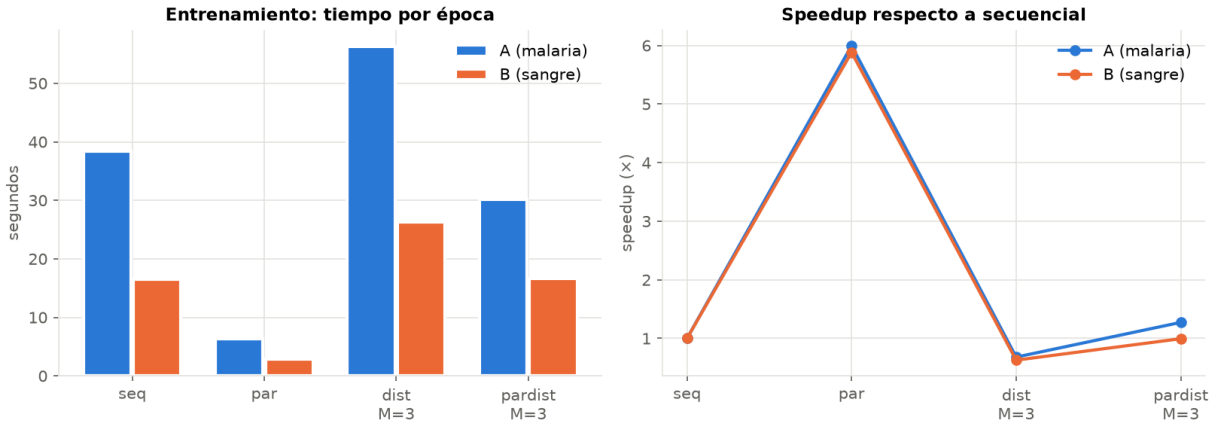


Figura 8: Tiempo por época y speedup del entrenamiento.

En una máquina, el entrenamiento paralelo es 6 veces más rápido. En cambio, distribuido por internet es **más lento que el secuencial**. En cada ronda de 10 pasos, cada laptop recibe y devuelve el modelo completo (unos 270 KB) pasando por el servidor en Santiago, y ese costo de comunicación supera lo que se gana repartiendo el cálculo. El paralelo distribuido lo compensa en parte, porque cada nodo calcula su ronda con 12 hilos. Como referencia, en una prueba previa sin red (3 contenedores en una sola laptop), el

distribuido 3×1 sí aceleraba 3,4× y el paralelo distribuido 5,7×. El accuracy baja a 80–83 % porque cada nodo ve un tercio de los datos entre promedios y solo hay 3 épocas.

4.3. Inferencia en las 4 configuraciones

Configuración	Nodos × núcleos	Throughput	Latencia media	Máx. cola A / B	Errores
Secuencial	1×1	33,2 tx/s	12 839 ms	418 / 380	0
Paralelo	1×12	52,5 tx/s	1 412 ms	0 / 0	0
Distribuido	3×1	43,5 tx/s	1 699 ms	1 / 3	0
Paralelo distribuido	3×12	54,1 tx/s	1 367 ms	0 / 0	0

Tabla 12: Inferencia: 2000 consultas por configuración (1000 por escritorio y 1000 por web). La penúltima columna es el máximo de mensajes en espera en `cola_modeloA` y `cola_modeloB`.

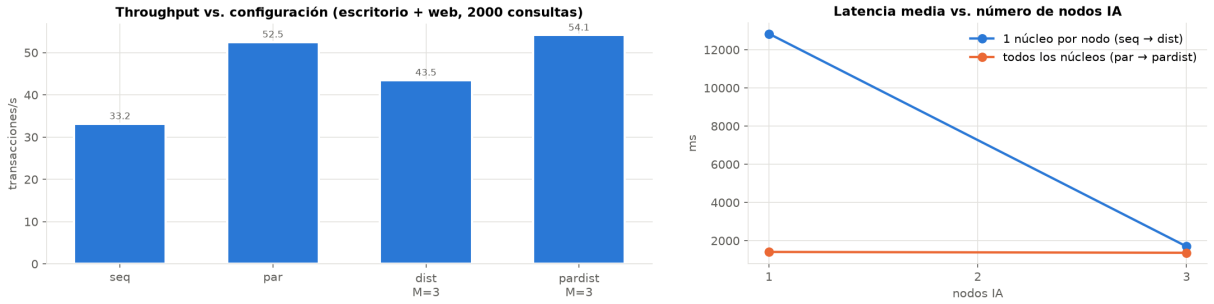


Figura 9: Throughput vs. configuración (izquierda) y latencia vs. número de nodos de IA (derecha).

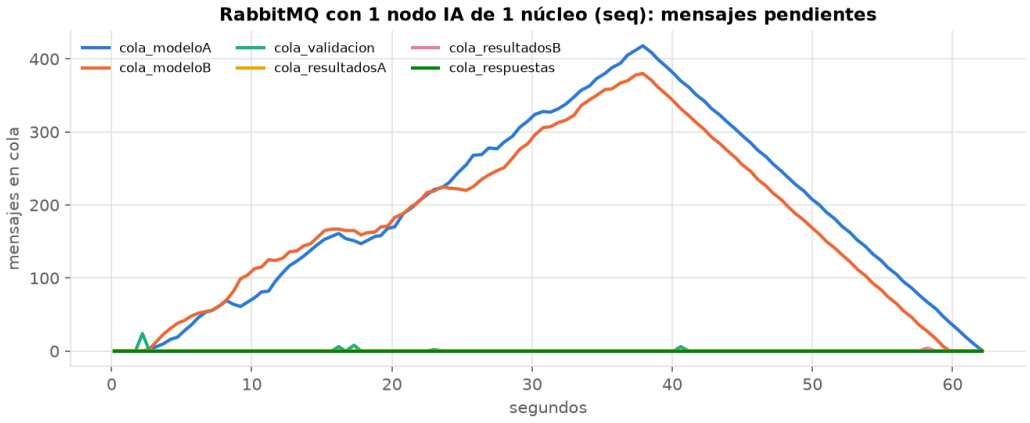


Figura 10: Colas de RabbitMQ con un solo nodo de IA de 1 núcleo: se acumulan los mensajes de los modelos.

En secuencial, `cola_modeloA/B` llega a unos 400 mensajes en espera y la latencia sube a 12,8 s (Figura 10). Sin embargo, el nodo de IA usa solo un 20 % de su núcleo: con 1 hilo, RabbitMQ le entrega un mensaje a la vez (*prefetch* 1), y cada mensaje espera un viaje de ida y vuelta a Santiago antes del siguiente. Con 12 hilos o con 3 nodos, varios mensajes viajan a la vez, las colas de los modelos quedan vacías y la latencia baja casi 10 veces. A partir de ahí el límite ya no es la IA: lo que más espera es `cola_validacion` (hasta 48 mensajes), porque BD2 valida cada consulta dos veces desde la casa de Integrante01. Por eso el throughput se estabiliza en 52–54 transacciones por segundo.

4.4. Prueba de saturación

Cliente	Transacciones	OK	Errores	Latencia media	p95	Throughput
Escritorio	1 000	1 000	0 %	2 107 ms	3 498 ms	16,7 tx/s
Web	1 000	1 000	0 %	2 116 ms	3 449 ms	16,6 tx/s
Android	1 000	1 000	0 %	2 590 ms	–	32,0 tx/s

Tabla 13: Saturación con los tres clientes a la vez y los 3 nodos de IA.

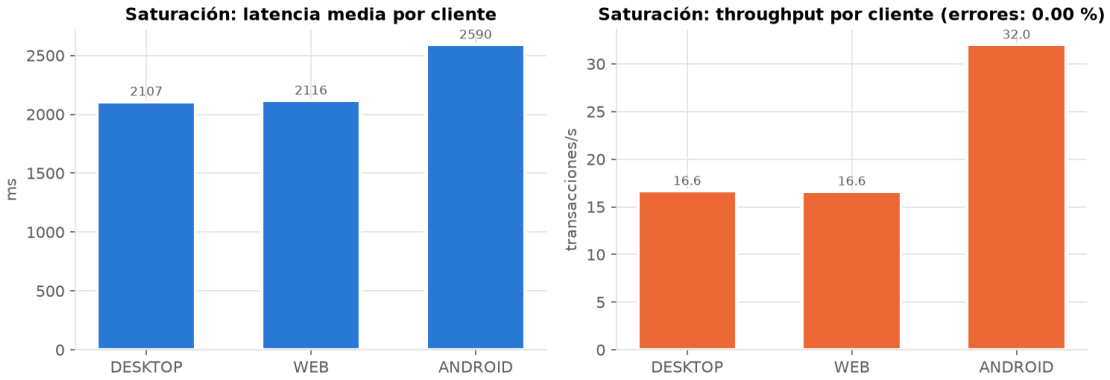


Figura 11: Latencia y throughput por cliente durante la saturación.

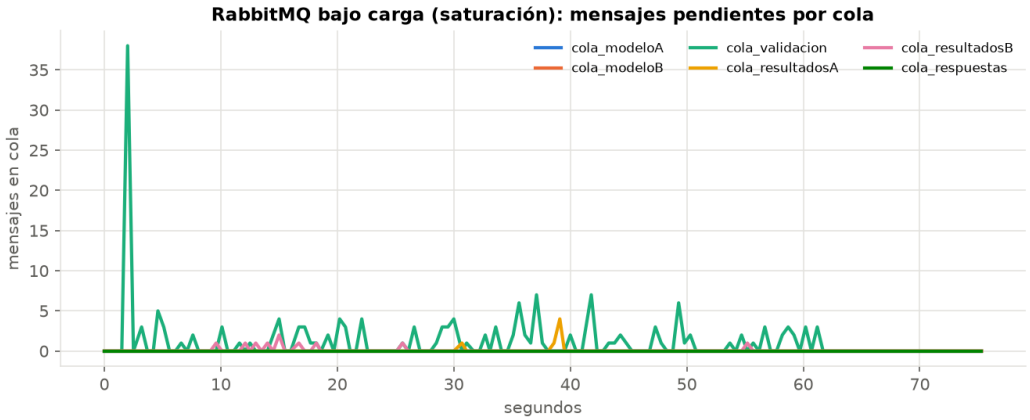


Figura 12: Uso de RabbitMQ bajo carga: mensajes pendientes por cola durante la saturación.

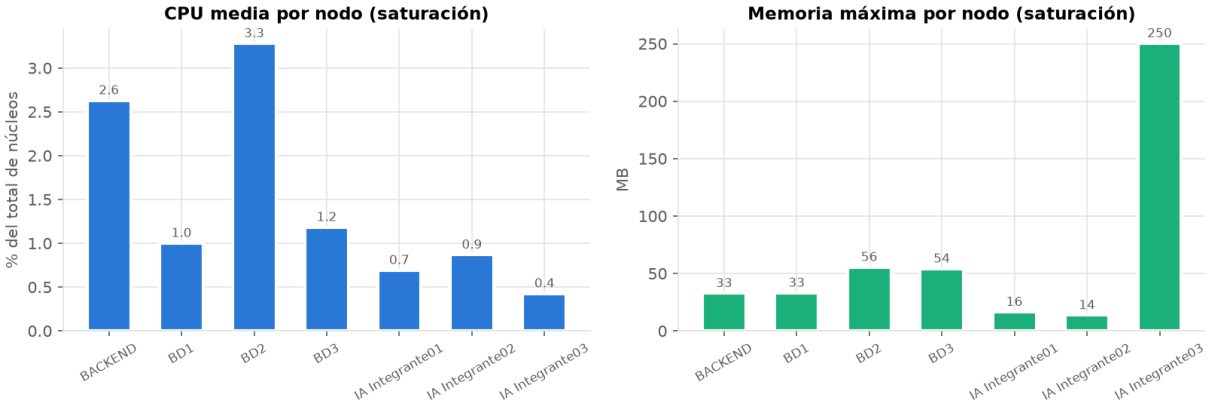


Figura 13: CPU (como porcentaje del total de núcleos de cada máquina) y memoria de cada nodo durante la saturación.

Las 3000 transacciones se respondieron sin errores. Con los 3 nodos de IA, las colas de los modelos no crecen; lo que fluctúa es `cola_validacion` (máximo 38 mensajes), que se vacía en cuanto baja la ráfaga. Ningún nodo pasa del 10 % de CPU: el sistema está limitado por la latencia de la red, no por el cómputo. Los valores de Android son los que mostró la app al terminar (Figura 2); su CSV se sobrescribió en una corrida posterior sin conexión, por lo que no hay p95. Un primer intento desde otro celular terminó con 0/1000 porque la app había perdido la conexión antes de empezar; esas consultas no llegaron a salir del teléfono.

4.5. 1000 registros y validación de las bases

El generador registró 1000 clientes nuevos aleatorios: 1000 OK, 116 ms de latencia media y 54 registros por segundo. Al final, `verificar_consistencia.py` cruzó las tres bases por RabbitMQ:

Chequeo	**Valor**
Resultados en BD1 (PostgreSQL, Integrante02) / BD3 (MySQL, Integrante03)	5 835 / 6 473
Validaciones de imagen válidas en BD2 (MariaDB, Integrante01)	12 308
Resultados sin validación en BD2 / en BD1 y BD3 a la vez / duplicados	0 / 0 / 0
Consultas OK para el cliente sin resultado guardado	0
Clientes en BD2 (1 004 iniciales + 1 000 registrados)	2 004
Resultado	**consistentes**

Tabla 14: Validación cruzada de BD1, BD2 y BD3 al final de las pruebas.

4.6. Discusión general

Los resultados muestran que el paralelismo funciona en una sola máquina, tanto al entrenar (6 veces más rápido) como al servir consultas (latencia casi 10 veces menor). La distribución por internet, en cambio, no acelera el entrenamiento: el costo de enviar el modelo en cada ronda supera al del cálculo. Es el costo de paralelización estudiado en el curso; para aprovechar la distribución convendría sincronizar menos seguido o tener los nodos en una red local. En la inferencia, RabbitMQ deja ver dónde está el cuello de botella en cada momento: primero en los nodos de IA y, al agregar hilos y nodos, en la validación de BD2.

También deben considerarse los límites de las mediciones: se hizo una corrida por configuración, sobre redes domésticas variables y con laptops de procesadores distintos, y la carga se lanzó desde una laptop que también corría BD1 y un nodo de IA. Las cifras describen el comportamiento del sistema completo más que un *microbenchmark* en condiciones idénticas.

5. Conclusiones

- Se implementó un sistema distribuido con clientes Android, de escritorio y web, un backend con sockets propios, RabbitMQ como único *middleware*, nodos de IA en Go y tres bases de datos heterogéneas. Funcionó de punta a punta con el servidor en Google Cloud y tres laptops en tres casas.

- Las CNN escritas desde cero, con *micro chunks* de convolución y de MLP, alcanzan 95,2 % en malaria y 88,6 % en 8 tipos de células sanguíneas.
- En una máquina, el entrenamiento paralelo es 6 veces más rápido. Distribuido por internet es más lento que el secuencial, porque la comunicación pesa más que el cómputo; sin red, el mismo esquema acelera 3,4 veces con 3 nodos.
- En la inferencia, pasar de 1 hilo a varios hilos o nodos baja la latencia casi 10 veces; después, el cuello de botella pasa a la validación en BD2.
- El sistema respondió 11 000 transacciones sin errores, incluida la saturación con los tres clientes a la vez. Las transacciones, `FOR UPDATE`, las inserciones idempotentes y una conexión por hilo mantuvieron consistentes las tres bases.

6. Anexo de código fuente

El código fuente completo se incluye en el archivo ZIP entregado con este informe y en el repositorio `https://github.com/dodo-UNI-2026-02/BorgAICube`. Los archivos principales son:

- `codigo/java/src/cubo/`: backend, servidor web, clientes de escritorio y generador de carga.
- `codigo/go/`: CNN, nodo de IA y entrenamiento en las 4 configuraciones.
- `codigo/python/`: nodos de BD, validador, monitor, verificación de consistencia y gráficas.
- `codigo/android/`: app Android.
- `codigo/web/`: chat web.
- `codigo/sql/`: scripts de creación de BD1, BD2 y BD3.

7. Anexo de documentación

Requisitos

- Java 17 o superior, Go 1.24 o superior y Python 3.10 o superior (con `pika`, `psycopg2` y `pymysql`).
- RabbitMQ, y en cada nodo de BD su motor: PostgreSQL, MariaDB o MySQL.
- Android Studio, solo para compilar la app.

Servidor (en Google Cloud)

```
curl -fsSL https://raw.githubusercontent.com/dodo-UNI-2026-02/BorgAICube/main/scripts/
    nube_preparar_vm.sh -o preparar.sh && bash preparar.sh
bash ~/BorgAICube/scripts/servidor_nube.sh
```

Nodos (en cada laptop)

```
$env:CUBO_RABBIT_URL = "amqp://cubo:CLAVE@34.176.215.27:5672/cubo"
python codigo\python\nodo_bd.py --rol BD2 --db-host 127.0.0.1     (BD1 o BD3 segun la
    laptop)
.\nodo.exe -datos datos
```

Clientes

```
java -cp "codigo/java/out;codigo/java/lib/*" cubo.desktop.ClienteDesktop --host
    34.176.215.27
http://34.176.215.27:8080
```

Mediciones

```
./scripts/equipo.sh seq | par | entrenar_local | pardist | entrenar_dist | saturacion |
    registros | dist | validar
python codigo/python/graficas.py --resultados resultados/equipo
```

Los resultados quedan como CSV en la carpeta `resultados/equipo/`, y las gráficas, como PNG en su subcarpeta `graficas/`.

Referencias

[1] CC4P1 – Programación Concurrente y Distribuida. *Práctica 03 2026-II: Sistema distribuido de CNN con chat multiplataforma y middleware RabbitMQ*. Universidad Nacional de Ingeniería.

[2] S. Rajaraman et al. *Pre-trained convolutional neural networks as feature extractors toward improved malaria parasite detection in thin blood smear images*. PeerJ, 6:e4568, 2018. Dataset: National Library of Medicine (NIH).

[3] J. Yang et al. *MedMNIST v2: A large-scale lightweight benchmark for 2D and 3D biomedical image classification*. Scientific Data, 10:41, 2023.

[4] A. Acevedo et al. *A dataset of microscopic peripheral blood cell images for development of automatic recognition systems*. Data in Brief, 30:105474, 2020.

[5] RabbitMQ. *RabbitMQ documentation: queues, consumer prefetch and RPC*. `https://www.rabbitmq.com/docs`.

[6] Material del curso CC4P1. *Desafío de los Sistemas Distribuidos*.

[7] Material del curso CC4P1. *Monitores, condición de carrera y thread-safe (Hilos_v12)*.

[8] Material del curso CC4P1. *Desempeño y optimización para la latencia (Desempeño251_v03)*.

[9] Google Cloud. *Compute Engine documentation*. `https://cloud.google.com/compute/docs`.